Hyeonjun Kil

Backend Engineer / Platform Part Leader

kboxstar@gmail.com GitHub ↗ LinkedIn ↗ Web Portfolio ↗

Flex — Constraint-Explicit Work Planner

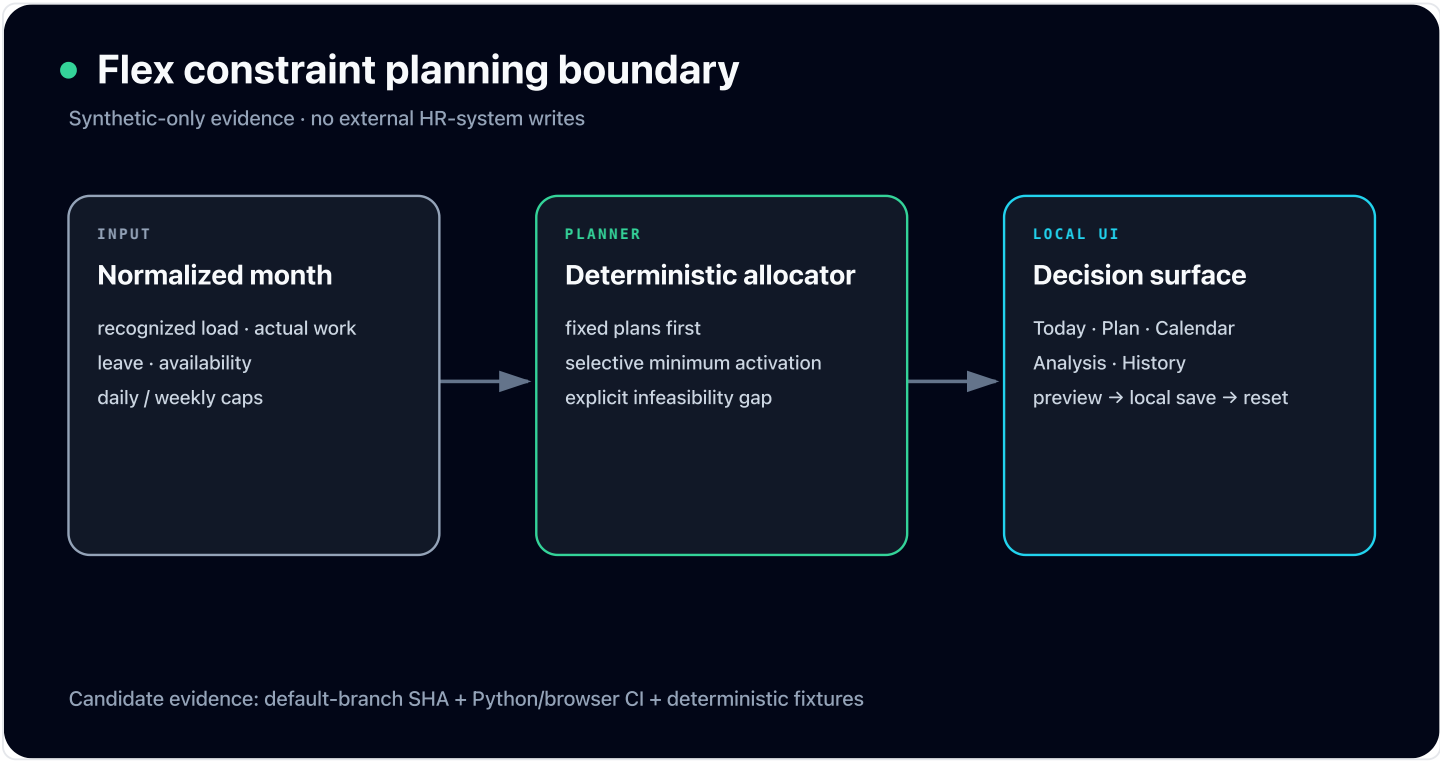
2026.07 — Present · Sole designer, implementer, and verifier · Personal

GitHub ↗ Detail ↗

Deterministic monthly allocator · per-date constraints · Python/React verification

A local-first work planner that combines recognized load, actual work, leave, and daily/weekly caps into one deterministic allocation contract, exposing infeasible targets as explicit gaps instead of forced over-allocation.

Python | React | TypeScript | FastAPI | Playwright | Property Testing | GitHub Actions



When a synthetic monthly target exceeds remaining daily and weekly capacity, the planner preserves those caps and returns insufficient_slots with the remaining gap.

Performance Metrics	Before	After
Infeasible state	Risk of forced allocation	**insufficient_slots + gap** (hard limits preserved)
Date-change scope	Risk of full-month overwrite	**fixed date + remainder redistribution** (preview and redistribution boundary)

Problem Solving

Forced allocation above remaining capacity would hide an infeasible plan

Apply daily/weekly caps and fixed plans first, compute allocatable time, then return the remaining gap as an explicit state

→ Preserves hard limits while displaying insufficient_slots and the remaining gap

A date override could unexpectedly overwrite the whole monthly plan

Lock the selected date first, preview conflicts, then deterministically redistribute only the remainder

→ Shows change scope and conflict boundaries in preview without external writes

Tech Decisions

- Restored the existing _minimum_activation flow so minimum work remains a selective activation floor rather than a mandatory seed
- Kept the decision surface reproducible through URL/local state without adding external-system integration

Highlights

- One allocator contract for fixed-plan priority, selective minimum activation, and hard-limit preservation
- Infeasibility exposed as a remaining gap and reason
- Date-override preview and redistribution calculation for remaining dates
- Synthetic scenarios, public-boundary scan, and Python/Playwright CI

Lessons Learned

- A planner must model infeasibility before optimizing, or it will distort reality.
- User overrides remain predictable when the fixed scope is separated from redistribution of the remainder.

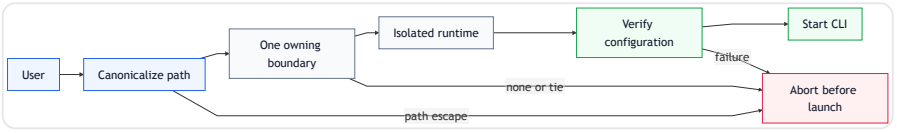
A Public Launcher for Isolated, Verified AI CLI Runtimes

2026.04 — Present · Sole designer, implementer, and operator · Personal

GitHub ↗ Demo ↗ v0.20.0 Release ↗ CI ↗ Detail ↗

Public launcher · 3 AI CLIs · contract demo

A public launcher that isolates global Claude Code, Codex, and Kiro state into project-scoped boundaries and blocks invalid workspaces or configuration drift before startup.



Performance Metrics	Before	After
Runtime isolation	Projects shared global CLI state	**Runtime homes per registered project** (Sessions, skills, MCP tools, and policy stay boundary-scoped)
Boundary resolution	Risk of profile inference from names or remotes	**One owning boundary from the canonical path** (Outside, escaped, or ambiguous paths stop before AI CLI startup)
Public verification	Evidence depended on a private environment	**Public launcher + contract demo + CI** (Core contracts and failure behavior reproduce without login)

Problem Solving

Sharing one global AI CLI home across projects could mix sessions, tools, and rules across trust boundaries.

Generated Codex and Kiro runtime homes per registered project and routed profile commands and harness-exec through the same execution policy.

→ Runtime state stays project-scoped, with installation, profile, and runtime behavior verified in the public repository suite.

A workspace manager that guessed profiles from names or remotes could start an agent with the wrong tools and permissions.

Made harness-auto canonicalize the current path and select only the single most-specific registered boundary, rejecting outside paths, symlink escapes, and equally specific duplicates.

→ Profile selection is based on path ownership rather than inference, and no AI CLI starts when ownership is unresolved.

A private operating environment alone did not let visitors independently verify profile isolation and source-verification contracts.

Linked the public harness-launcher implementation and tests to a separate public contract demo. The demo checks profile uniqueness, path escape, source SHA-256 drift, and relationship integrity with the standard library.

→ The core boundaries and failure behavior are reproducible from README, source, and CI without login, while private company data and production indexes remain outside public evidence.

Concert Reservation Service

2024.10 — 2024.11 · Personal

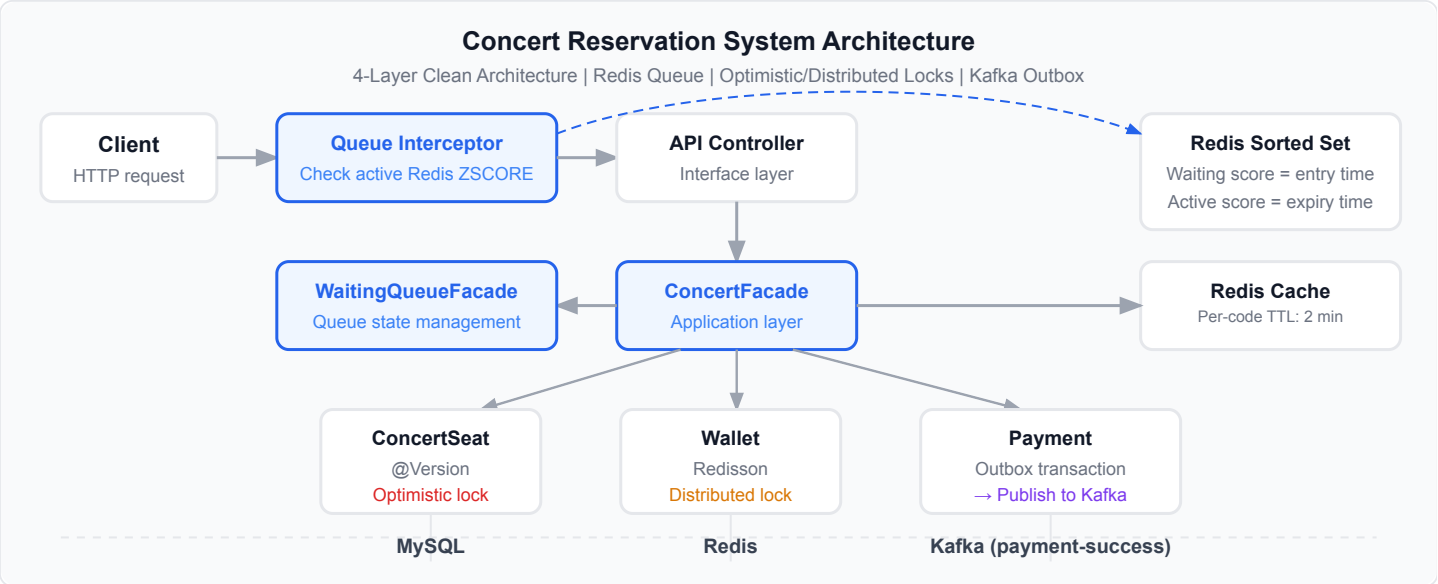
GitHub ↗ Detail ↗ JPA Pessimistic Lock, Optimistic Lock and Retry ↗ Spring Boot Concert Reservation Concurrency Analysis ↗

Distributed Lock Implementation with Spring Boot Redis ↗ Cache and Caching Strategy ↗

Configured 3,000 requests/s · 2,400 max VUs · observed reservation error rate 0.64% · Prometheus+Grafana

Concert reservation project that verifies same-seat booking and wallet payment with optimistic and Redisson locks, then reduces cache-stampede p95 from 5.74s to 676ms through cache warming and TTL changes in the public load-test report

Java Spring Boot JPA Redis MySQL Kafka Docker K6



Performance Metrics	Before	After
Reservation p95	5.74s	**676ms** (cache warming + TTL 2m→5m)
Reservation error rate	-	**0.64%** (394 failures in the reservation scenario)
Queue p95	-	**62.5ms** (0% error rate in the queue scenario)

Problem Solving

Duplicate-reservation risk when 100 threads target the same seat

Exercised the conflict path with ConcertSeat @Version optimistic locking and a 100-thread Spring integration test

→ Assertions confirm one reservation row and the reserved state after the test

Balance integrity corruption under concurrent point charging and payments

Serialized per-user charge and payment paths with a Redisson distributed lock (userWalletLock:{userId})

→ A 100-thread charge test compares the final balance with the expected sum, while the payment test checks a single payment state

Observed a cache stampede and p95 5.74s latency during the reservation load test

Applied cache warming three minutes before reservations and changed TTL from two to five minutes

→ Public report records reservation p95 5.74s → 676ms, within its configured 1s SLO

Payment-event persistence, publication, and failed-event retry states needed separate boundaries

Kafka Transactional Outbox stores a PENDING outbox_event within the payment transaction, then a scheduler publishes it asynchronously

→ Separated PENDING outbox creation from publication-failure state and retry paths in code and tests

Tech Decisions

- @Version optimistic locking for seats: a 100-thread same-seat integration test asserts one reservation row
- Per-user Redisson locking for payment and charging: 100-thread tests compare payment state and final balance with expected values
- Redis Sorted Set queue: scores track entry order and expiry state
- Create a PENDING outbox record in the payment transaction, then let the scheduler manage publication-failure and retry paths

Highlights

- One reservation row after a 100-thread same-seat integration test
- Per-user Redisson serialization for wallet and payment paths
- Public-report p95 5.74s → 676ms after cache warming and TTL changes
- PENDING outbox in the payment transaction plus scheduler publication and retry paths
- Configured 3,000 requests/s and 2,400 max VUs kept separate from observed outcomes

Lessons Learned

- Separated optimistic locking for same-seat contention from per-user distributed locking for wallet contention
- Kept configured k6 rate and max VUs distinct from observations reported by the load test

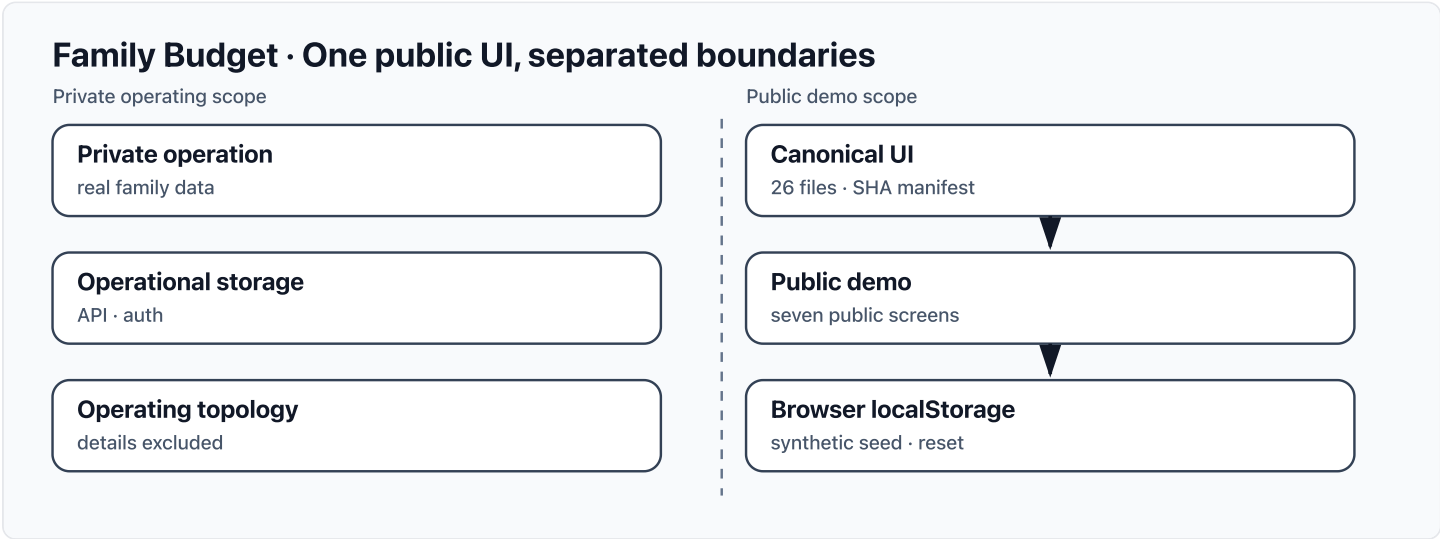
A Public Budget Demo Reproducing Seven Screens Without Exposing Family Data

2026.08 — 2026.08 · Personal

Live Demo ↗ GitHub ↗ Detail ↗

Pinned 26 public UI files with a hash manifest and connected synthetic seed data to a browser-local adapter, reproducing entry, read-back, and reset flows across seven public screens without real family data, an operating database, or credentials.

TypeScript | Next.js | React | Vitest | Vercel



Verified outcomes	Before	After
Canonical public UI	Drift risk from separately rewritten screens	**26 files** (Public files pinned by a SHA-256 manifest)
Public product flow	Hard to demonstrate without operational data or authentication	**7 screens** (Synthetic entry, read-back, and reset flow)
Release verification	A successful build alone cannot prove the UI boundary	**33 tests** (Plus production build and boundary, runtime, and CSS checks)

Operating limits

- Data remains in the current browser; there is no cross-device sync, user account, sharing, or server backup.
- The public demo does not prove the operational backend, real data scale, or multi-user collaboration.
- The public-boundary scan checks defined source trees and literal patterns; it is not a repository-wide general secret scanner or a WCAG audit.

Public evidence

Manifest ↗ Adapter ↗ Runtime ↗

Additional Backend Projects

Gamebang — Server-Authoritative Realtime Game Room (Personal) — TypeScript, React, WebSocket, Cloudflare Workers Detail ↗

A five-game realtime service where mobile clients request actions and a room-scoped Durable Object owns phase, turn, hidden information, and reconnect behavior.

Clinical-Lab Job Decision Dashboard (Personal) — Python, FastAPI, SQLite, React Detail ↗

A mobile-first dashboard that normalizes job-platform and hospital-career-page sources, explains deadline/region/employment recommendations, and keeps account-scoped application state.

Daesin Logistics Dispatch Operations Ledger (Personal) — TypeScript, Express, Next.js, React Detail ↗

A current-day dispatch ledger for route, vehicle, and statistics views, with singleton collection ownership and a fixed browser data boundary

NyamNyam WeDu — Cafeteria Menu Alert Bot (Personal) — TypeScript, Playwright, Slack Bot API, Express Detail ↗

Bot that crawls weekly cafeteria menus from Kakao Channel via Playwright and auto-delivers to Slack — Clean Architecture + TSyringe DI, Slack Bot API slash command, up to 6 retries, DeliveryHistory duplicate prevention

StartupPool (Team) — NestJS, TypeScript, PostgreSQL, JavaScript Detail ↗

Backend developer in 8-member cross-functional team building a startup vertical network platform — designed and implemented core community APIs on NestJS, integrated 4 OAuth 2.0 social login providers, built independent admin backend across 2-month sprints